Phase 3 Documentation

Formal Proof in Lean 4

Chosen Theorem: Expression Determinism

Authors: Ehsan Soveizi & Mahdi Rashidi

Project item	Final version
Project phase	Phase 3 - Formal Proof
Proof assistant	Lean 4
Chosen theorem	Expression Determinism
Semantic model	Relational big-step evaluation of expressions
Proof method	Induction on the first derivation and cases on the second derivation
Core claim	The same expression evaluated in the same state cannot produce two different values.

Document purpose

This updated report explains the direct relational proof of Expression Determinism. It removes the former function-based bridge and all statement-level material. Focused Lean snippets are included only where they clarify the rules or the proof.

Contents

1. Executive Summary
2. Modeled Expression Subset and Rationale
3. Theorem Selection and Formal Meaning
4. Minimal Lean Background for Reading the Proof
5. Formal Model of Expressions
6. Relational Evaluation Rules: EvalExpr
7. Direct Proof Strategy
8. Case-by-Case Proof of expression_determinism
9. Worked Derivation Example
10. Why This Proof Establishes Determinism
11. Compilation, Limitations, and Presentation Notes

How to use this report

For presentation, begin with Sections 1-3 to state the scope and theorem. Use Sections 5-6 to explain the formal rules. Spend most of the proof explanation on Sections 7-8, because they show induction on the first derivation, dependent cases on the second derivation, and the use of the induction hypotheses.

1. Executive Summary

Phase 3 moves from model checking particular generated programs to proving a mathematical property of the language semantics itself. The selected property is Expression Determinism, formalized and proved in Lean 4.

The theorem states that relational expression evaluation has at most one result. If the same expression is evaluated in the same program state by two valid derivations, then the two produced integer values are equal. This shows that the operational rules do not assign conflicting meanings to one expression-state pair.

For example, suppose the state stores x = 5 and the expression is x + 2. One derivation may be written independently from another, but both must use the variable rule for x, the numeric rule for 2, and the addition rule for the complete expression. Therefore both derivations conclude the value 7.

Main idea in one sentence
Analyze the first EvalExpr derivation by induction, inspect the second derivation with cases, and use the induction hypotheses to prove equality of corresponding operand values.

2. Modeled Expression Subset and Rationale

The Lean model intentionally uses a compact, total subset of Hash expressions: numeric literals, variables, addition, multiplication, less-than comparison, and equality comparison. This subset contains base cases, recursive arithmetic, and boolean-like comparisons while avoiding unrelated statement semantics.

Expression constructor	Hash-level idea	Reason for inclusion
Expr.num n	Integer literal such as 5	Base case with no premises.
Expr.var x	Variable such as x	Connects expressions to the state.
Expr.add e1 e2	e1 + e2	Representative recursive arithmetic rule.
Expr.mul e1 e2	e1 * e2	Second arithmetic case using the same proof pattern.
Expr.lt e1 e2	e1 < e2	Comparison returning 1 or 0.
Expr.eq e1 e2	e1 == e2	Equality comparison returning 1 or 0.

2.1 Why Statement Semantics Is Not Included

The selected theorem concerns Expr and EvalExpr only. Definitions such as Stmt, update, statement-level BigStep, assignment, sequencing, if, while, and for-loop desugaring do not appear in the statement or proof. Keeping them would enlarge the file without contributing to Expression Determinism.

EvalExpr is already a relational big-step semantics, but it is a big-step semantics for expressions: it relates a complete expression and a state directly to a final integer value. A separate relation named BigStep is only needed when proving properties of complete statements and final states.

Scope statement for instructors
The theorem proves determinism for the six expression constructors represented in Expr. It does not claim statement determinism and does not require statement syntax.

3. Theorem Selection and Formal Meaning

In mathematical notation, the selected property is:

```
<e, s> evaluates to v1  and  <e, s> evaluates to v2  implies  v1 = v2
```

Here e is an expression, s is one fixed state, and v1 and v2 are candidate integer results. The two derivations may be distinct proof objects, but they start from exactly the same expression and exactly the same state.

```
theorem expression_determinism :
  forall {e : Expr} {s : State} {v1 v2 : Int},
    EvalExpr e s v1 ->
    EvalExpr e s v2 ->
    v1 = v2 := by
```

The theorem is a uniqueness theorem. It says that if two evaluations exist, their results agree. It does not separately prove that every expression has an evaluation derivation; existence and uniqueness are different properties.

4. Minimal Lean Background for Reading the Proof

Lean construct	Meaning in this proof
abbrev	Creates a short type name: Var is String and State is Var -> Int.
inductive	Defines syntax or a logical relation through constructors.
Prop	The universe of propositions; EvalExpr e s v is a proposition.
intro	Moves quantified variables or assumptions into the local context.
induction hFirst	Creates one proof branch for every constructor that could build hFirst.
generalizing v2	Keeps the competing result arbitrary in the induction hypotheses.
cases hSecond	Inspects the last rule of the second derivation and removes impossible cases.
rw [ha, hb]	Rewrites operand values using equalities established by induction.
rfl	Closes a reflexive equality such as n = n.

5. Formal Model of Expressions

5.1 Variables and States

```
abbrev Var := String
abbrev State := Var -> Int
```

A state is modeled as a function. For a state s and variable name x, the application s x is the integer stored for x. Expression evaluation only reads from the state, so no update function is required.

Example state

If s "x" = 5 and s "y" = 10, then Expr.var "x" evaluates to 5 and Expr.var "y" evaluates to 10.

5.2 Expression Syntax

```
inductive Expr where
  | num : Int -> Expr
  | var : Var -> Expr
  | add : Expr -> Expr -> Expr
  | mul : Expr -> Expr -> Expr
  | lt  : Expr -> Expr -> Expr
  | eq  : Expr -> Expr -> Expr
 deriving Repr
```

All expression results use the type Int. Comparison results are encoded as 1 for true and 0 for false. This keeps the relation uniform: EvalExpr always relates an expression and state to an integer.

6. Relational Evaluation Rules: EvalExpr

EvalExpr is not an executable evaluator. It is an inductive proposition. A value h : EvalExpr e s v is evidence, or a derivation tree, showing that expression e evaluates to v in state s.

```
inductive EvalExpr : Expr -> State -> Int -> Prop where
```

Function versus relation

A function computes one output by definition. A relation may in principle permit several outputs. The purpose of expression_determinism is to prove that these particular relational rules permit at most one output.

6.1 Literals and Variables

```
| num : forall (n : Int) (s : State),
    EvalExpr (Expr.num n) s n

| var : forall (x : Var) (s : State),
    EvalExpr (Expr.var x) s (s x)
```

The num and var rules are axioms because they have no evaluation premises. A literal evaluates to itself. A variable evaluates to the value stored in the current state.

6.2 Arithmetic Expressions

```
| add : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
    EvalExpr e1 s v1 ->
    EvalExpr e2 s v2 ->
    EvalExpr (Expr.add e1 e2) s (v1 + v2)

| mul : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
    EvalExpr e1 s v1 ->
    EvalExpr e2 s v2 ->
    EvalExpr (Expr.mul e1 e2) s (v1 * v2)
```

These rules are compositional. The conclusion for the complete expression can only be formed after derivations for the left and right operands have been supplied. The proof therefore receives one induction hypothesis for each operand derivation.

6.3 Comparison Expressions

```
| lt : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
    EvalExpr e1 s v1 ->
    EvalExpr e2 s v2 ->
    EvalExpr (Expr.lt e1 e2) s
      (if v1 < v2 then 1 else 0)

| eq : forall (e1 e2 : Expr) (s : State) (v1 v2 : Int),
    EvalExpr e1 s v1 ->
    EvalExpr e2 s v2 ->
    EvalExpr (Expr.eq e1 e2) s
      (if v1 = v2 then 1 else 0)
```

The comparison result is determined by the two operand values. Once the proof shows that corresponding operand values in the two derivations are equal, the two if-expressions become identical.

7. Direct Proof Strategy

The proof compares two derivations directly. It does not introduce an executable evalExpr function and does not prove an auxiliary soundness theorem. All reasoning is based on the constructors of EvalExpr.

```
intro e s v1 v2 hFirst
induction hFirst generalizing v2 with
```

After intro, hFirst is the first derivation. Induction on hFirst analyzes the last rule used in that derivation and recursively provides induction hypotheses for its premise derivations.

7.1 Why Induction Is Performed on hFirst

The recursive structure required by the proof is stored in the derivation, not merely in the syntax tree. In the add case, hFirst contains two smaller derivations h1 and h2. Lean therefore generates ih1 and ih2, which state that those subevaluations are deterministic.

7.2 Why v2 Is Generalized

The second derivation may use arbitrary intermediate operand values. The induction hypotheses must therefore apply to any competing result, not to one result fixed before induction. The phrase generalizing v2 makes each induction hypothesis sufficiently general.

```
ih1 : forall {b1 : Int}, EvalExpr e1 s b1 -> a1 = b1
ih2 : forall {b2 : Int}, EvalExpr e2 s b2 -> a2 = b2
```

7.3 Why cases Is Performed on hSecond

In every induction branch, hSecond has the same outer expression as hFirst. Dependent case analysis determines which constructor could have produced hSecond. If the expression is Expr.add e1 e2, constructors such as num, var, mul, lt, and eq are impossible, so only the add case remains.

Proof plan
Base rules close by reflexivity. Compound rules compare the two operand derivations using the induction hypotheses and then rewrite the complete result.

8. Case-by-Case Proof of expression_determinism

8.1 Numeric Literal Case

```
| num n s =>
  intro hSecond
  cases hSecond
  rfl
```

The first derivation proves EvalExpr (Expr.num n) s n. The second derivation has the same expression, so cases hSecond leaves only the num constructor. Both results are n, and the remaining goal n = n is closed by rfl.

8.2 Variable Case

```
| var x s =>
  intro hSecond
  cases hSecond
  rfl
```

Both derivations evaluate the same variable x in the same state s. The only applicable rule returns s x. After cases, the goal is s x = s x.

8.3 Addition Case

```
| add e1 e2 s a1 a2 h1 h2 ih1 ih2 =>
  intro hSecond
  cases hSecond with
  | add _ _ _ b1 b2 h1' h2' =>
    have ha : a1 = b1 := ih1 h1'
    have hb : a2 = b2 := ih2 h2'
    rw [ha, hb]
```

The first derivation evaluates e1 to a1 and e2 to a2, so its final result is a1 + a2. The second add derivation evaluates the same operands to b1 and b2, so its result is b1 + b2.

Applying ih1 to h1' produces ha : a1 = b1. Applying ih2 to h2' produces hb : a2 = b2. Rewriting with both equalities changes the goal into b1 + b2 = b1 + b2.

```
Before rewriting:  a1 + a2 = b1 + b2
After ha and hb:  b1 + b2 = b1 + b2
```

8.4 Multiplication Case

```
| mul e1 e2 s a1 a2 h1 h2 ih1 ih2 =>
  intro hSecond
  cases hSecond with
  | mul _ _ _ b1 b2 h1' h2' =>
    have ha : a1 = b1 := ih1 h1'
    have hb : a2 = b2 := ih2 h2'
    rw [ha, hb]
```

The multiplication branch has the same recursive proof shape as addition. The two induction hypotheses establish equality of the left operand values and equality of the right operand values. Rewriting makes the two products identical.

8.5 Less-Than Case

```
| lt e1 e2 s a1 a2 h1 h2 ih1 ih2 =>
  intro hSecond
  cases hSecond with
  | lt _ _ _ b1 b2 h1' h2' =>
    have ha : a1 = b1 := ih1 h1'
    have hb : a2 = b2 := ih2 h2'
    rw [ha, hb]
```

The goal initially compares two integer-encoded comparisons:

```
(if a1 < a2 then 1 else 0)
=
(if b1 < b2 then 1 else 0)
```

The induction hypotheses prove a1 = b1 and a2 = b2. After rewriting, the conditions are identical, so both if-expressions choose the same branch and return the same integer.

8.6 Equality Case

```
| eq e1 e2 s a1 a2 h1 h2 ih1 ih2 =>
  intro hSecond
  cases hSecond with
  | eq _ _ _ b1 b2 h1' h2' =>
    have ha : a1 = b1 := ih1 h1'
    have hb : a2 = b2 := ih2 h2'
    rw [ha, hb]
```

This case is identical in structure to less-than. Once both operand values are shown equal, the two equality tests are the same and therefore both evaluations return the same 1-or-0 value.

9. Worked Derivation Example

Consider the expression (x + 3) * 2 in a state s where s "x" = 4. A relational derivation is constructed from the leaves upward:

```
EvalExpr (Expr.var "x") s 4
EvalExpr (Expr.num 3) s 3
------------------------------ add
EvalExpr (Expr.add (Expr.var "x") (Expr.num 3)) s 7

EvalExpr (Expr.num 2) s 2
------------------------------------------------------ mul
EvalExpr
  (Expr.mul
    (Expr.add (Expr.var "x") (Expr.num 3))
    (Expr.num 2))
  s
  14
```

Suppose a second derivation claims a final value v. Because the outer expression is multiplication, the second derivation must end with the mul rule. The left induction hypothesis forces its left result to be 7, and the right induction hypothesis forces its right result to be 2. Therefore v = 7 * 2 = 14.

Derivation part	Why its result is unique
Expr.var "x"	The var rule always returns s "x", which is 4.
Expr.num 3	The num rule returns the literal 3.
x + 3	Unique operand values determine the unique sum 7.
Expr.num 2	The num rule returns the literal 2.
(x + 3) * 2	Unique operand values determine the unique product 14.

9.1 The Role of the Derivation Tree

The theorem does not compare textual source programs or execution traces. It compares proof trees. Each branch in the Lean induction corresponds to the last inference rule of the first tree, and cases hSecond reveals the last rule of the second tree.

Key structural fact
For this expression semantics, the outer syntax uniquely determines the only possible final evaluation rule. Recursive ambiguity could only come from the operands, and the induction hypotheses eliminate that ambiguity.

10. Why This Proof Establishes Determinism

The proof is exhaustive because EvalExpr has exactly six constructors and the induction contains exactly six branches. Every possible first derivation is therefore covered.

Constructor	Possible final rule of second derivation	How equality is proved
num	num only	Reflexivity
var	var only	Reflexivity
add	add only	Two induction hypotheses and rewriting
mul	mul only	Two induction hypotheses and rewriting
lt	lt only	Two induction hypotheses and rewriting
eq	eq only	Two induction hypotheses and rewriting

The proof is not circular. ih1 and ih2 apply only to premise derivations, which are structurally smaller than the current derivation. This is precisely the well-founded recursive reasoning justified by induction on an inductive proof object.

The proof also does not obtain determinism from Lean functions. There is no evalExpr function, no evalExpr_sound theorem, and no equality chain through a pre-existing function result. Determinism is derived solely from the shape of the inference rules.

Direct relational conclusion
Any two EvalExpr derivations for the same expression and state must agree at every corresponding subexpression and therefore agree on the complete result.

10.1 Why h1 and h2 Are Not Used Directly

In a recursive branch, h1 and h2 are the first derivation's premise proofs. The induction tactic has already used them to generate ih1 and ih2. The proof body therefore applies the induction hypotheses to h1' and h2', the corresponding premises from the second derivation.

10.2 What the Theorem Does Not Prove

Expression Determinism proves at most one result. It does not by itself prove that at least one result exists for every expression. A separate totality theorem would prove existence. Together, totality and determinism would establish exactly one result.

11. Compilation, Limitations, and Presentation Notes

11.1 Compilation

The source file is self-contained and requires no statement definitions or external semantic model. It can be checked from a terminal with:

```
lean ExpressionDeterminism.lean
```

Successful Lean compilation normally produces no terminal output. An error message indicates a syntax mismatch, a constructor pattern mismatch, or an unfinished proof goal.

11.2 Limitations and Extensions

- The theorem covers the selected constructors num, var, add, mul, lt, and eq.
- All expression results are integers; true is encoded as 1 and false as 0.
- Expression evaluation is pure and reads, but does not modify, the state.
- Division is omitted because division by zero requires an explicit error or partiality design.
- Function calls, fields, arrays, exceptions, and I/O require additional semantic structures.
- A new total binary operator can usually be added with one Expr constructor, one EvalExpr rule, and one analogous determinism branch.

11.3 Presentation Script

Short oral explanation

We modeled expression evaluation as the inductive relation EvalExpr. To prove determinism, we assume two derivations for the same expression and state. We induct on the first derivation and use cases on the second. The numeric and variable cases are reflexive. In every compound case, the induction hypotheses prove that corresponding operand values are equal, and rewriting proves that the complete results are equal.

Possible question	Suggested answer
Why is evalExpr not used?	A function is deterministic by definition. The required proof must show that the relational evaluation rules themselves are deterministic.
Why is there no Stmt or BigStep definition?	The selected theorem concerns only expressions and integer results. Statement syntax and state transitions are irrelevant to this proof.
Why induction on hFirst instead of e?	hFirst contains the recursive premise derivations and produces exactly the induction hypotheses needed for them.
Why generalize v2?	The induction hypotheses must apply to any competing result produced by the second derivation.
Why cases on hSecond?	It reveals the final rule of the second derivation and eliminates constructors incompatible with the outer expression.
Why does rw finish the recursive cases?	After the operand values are rewritten as equal, both complete result expressions are identical.

Final presentation emphasis

The central contribution is not the amount of code. It is the direct structural argument over two relational derivations: induction on the first, cases on the second, and recursive uniqueness of the operands.